Analysis of DLL side-loading attack on Google Updater

Ahmed Mahfooz Ali Khan BTHBI338

Perform the analysis of the code. What is the purpose of the program?

Retrieving Special Folder Path:

The program calls SHGetSpecialFolderPathW, a Windows API function that retrieves the path of a special folder (such as Desktop, Documents, etc.). This path is stored in a variable named pszPath.

String Manipulation:

It utilizes _wcscpy_s and _wcscat_s, which are safe versions of string copy and concatenation functions for wide-character strings, indicating that the program works with Unicode. The code constructs file paths by concatenating various components, including the folder path obtained earlier, along with specific file names (e.g., rundll32.exe, Goopdate.dll, and GoogleTool.exe).

Path Construction:

The program builds paths for these executables, likely intending to run or manipulate them later, concatenating strings with the "\" directory separator to form complete paths.

Data Manipulation:

A loop in the code manipulates bytes in an array (e.g., byte_100CB30, byte_100CB31, etc.). This loop swaps values, likely for data obfuscation or transformation.

Event Simulation:

The final line calls keybd_event, which simulates keyboard events. This technique is often used in malware to automate tasks by sending keystrokes to applications without user interaction.

Overall Purpose:

The program likely aims to:

- Construct paths to specific executable files (possibly related to Google software).

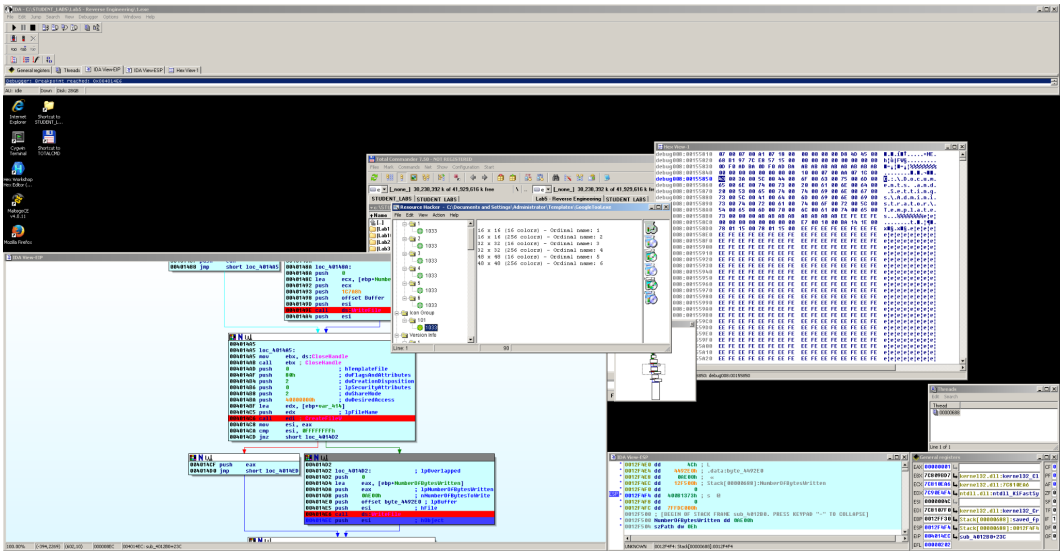


Figure 1: Reverse engineering view in IDA Pro showing dynamic path construction and memory manipulation by GoogleTool.exe and associated DLLs.

- Manipulate data in memory, which could serve for obfuscation or configuration.
- Simulate user input through keyboard events, potentially to launch applications or perform actions in other windows.

Overall, this code snippet appears to be part of an application that automates or interacts with Google-related software, exhibiting characteristics typical of potentially unwanted programs (PUPs) or malware. The specific files and paths used may imply a connection to software updates or background processes.

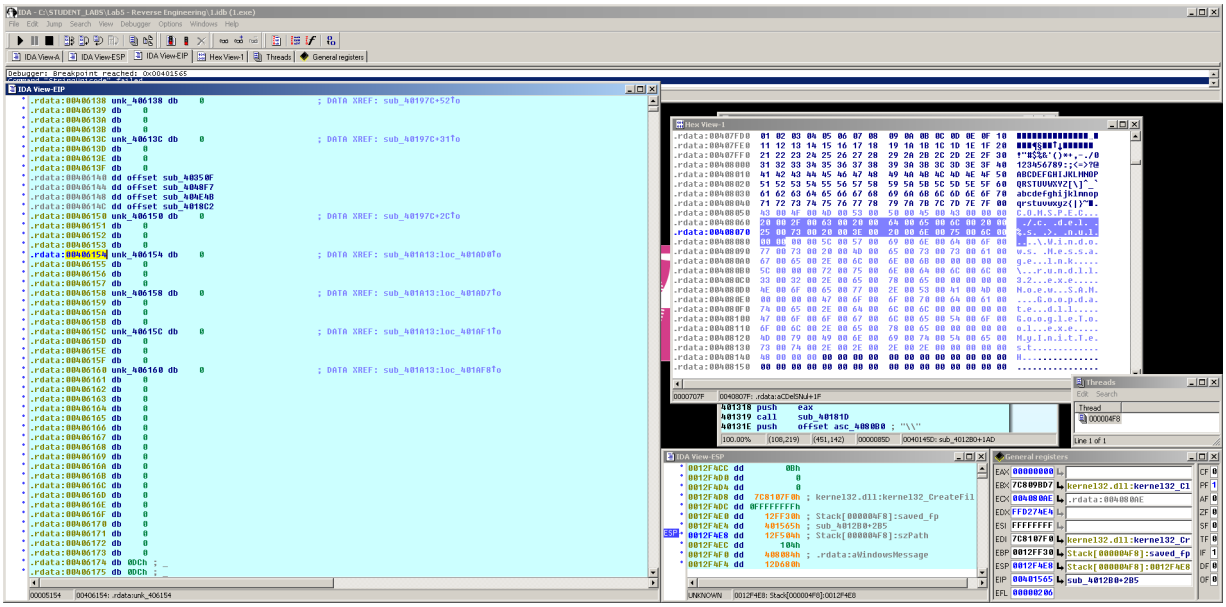
Determine the type (class) of malware.

Based on the provided code analysis, the characteristics suggest that this program could be classified as potentially unwanted software (PUP) or malware, specifically falling into one of the following categories:

- **Adware:** If it primarily aims to display advertisements or redirect users to certain websites.
- **Trojan:** Given its use of rundll32.exe and keyboard event simulation, it may execute other malicious payloads or facilitate unauthorized access, exhibiting Trojan-like behavior.

- **Spyware:** If it collects user data or monitors user activity without consent, especially if bundled with other software.
- **Rootkit:** If it employs techniques to hide its presence on the system, though this isn't explicitly indicated in the code snippet.

Summary:
The manipulation of file paths, keyboard event simulation, and potential execution of other binaries aligns it more closely with Trojan malware or adware. The exact classification would depend on further context about its behavior during execution and its specific functionalities.

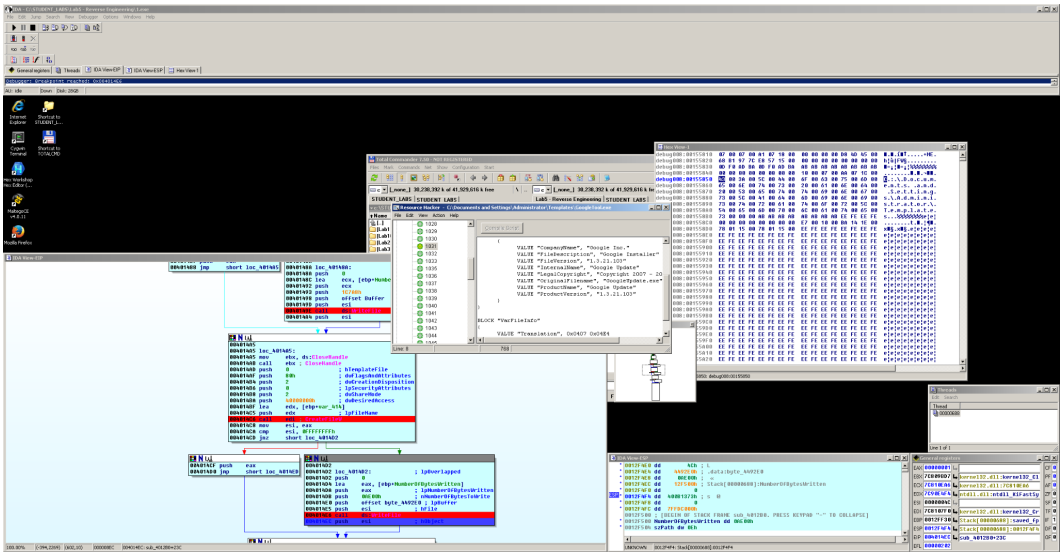


Describe the files dropped. What is the purpose of the dropped files?

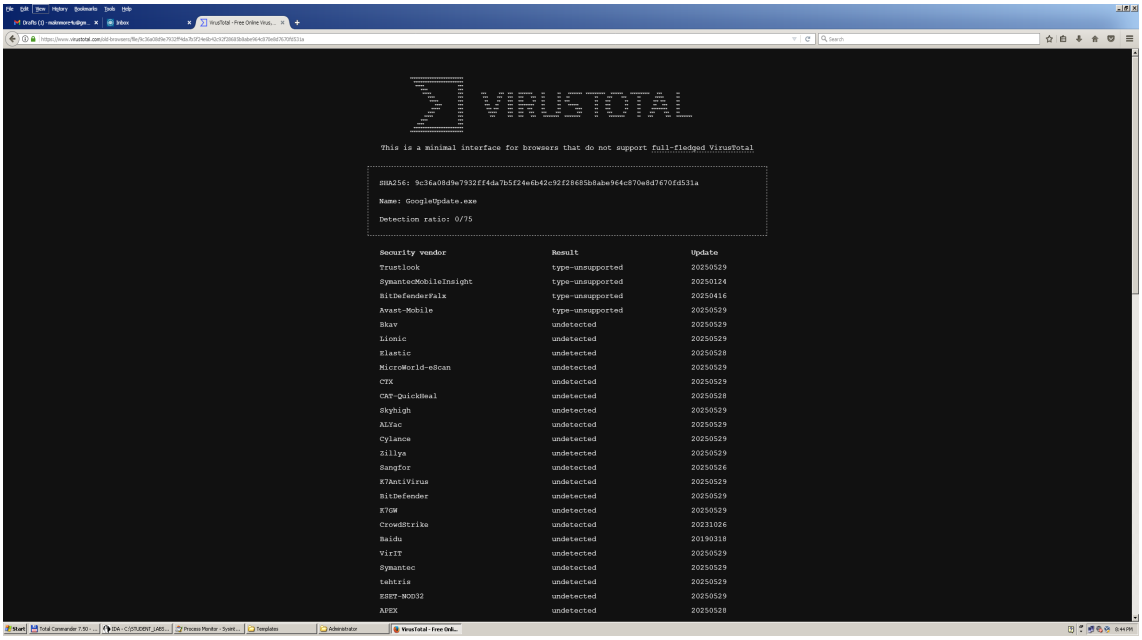
Here's a breakdown of the key files and their potential purposes:

- **rundll32.exe:**
    - **Purpose:** A legitimate Windows system file used to execute functions in DLL files. However, malware often uses it to run malicious code embedded in DLLs without raising suspicion. In this context, it could execute other dropped malicious files.

- **Goopdate.dll:**
    - **Purpose:** A malicious DLL created to exploit DLL side-loading by mimicking a legitimate Google DLL.
- **GoogleTool.exe:**
    - **Purpose:** A legitimate executable associated with google update . it is not malicious itself. Attackers placed a malicious goopdate.dll in the same folder. It hold some of the information regarding ads pop up.



and it is not malicious as there is no vulnerabilities found in it



This is a minimal interface for browsers that do not support full-fledged VirusTotal

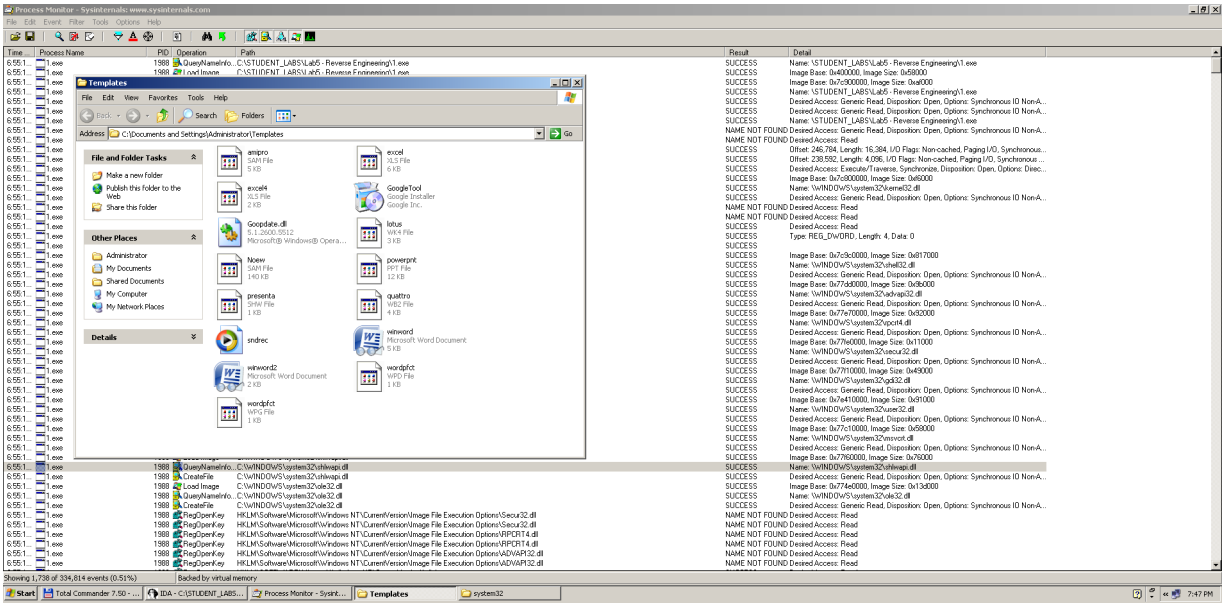
SHA256: 9e36a08d9e7932ff44a7b5f24e6b42c92f2868Sb8abe964c870e8d7670fd531a

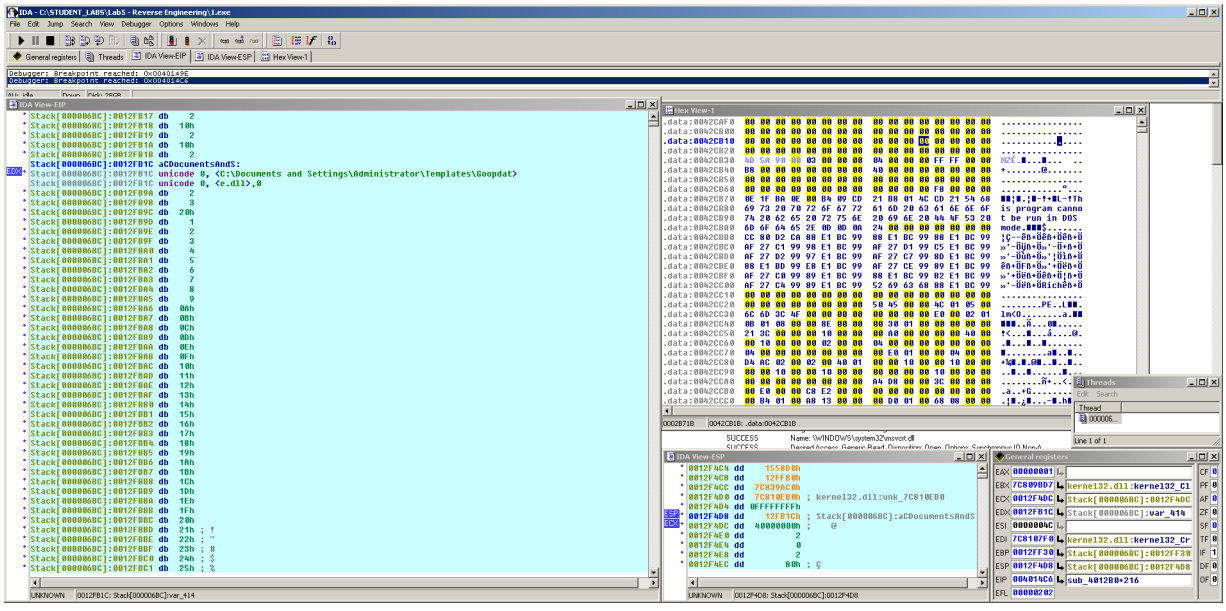
Name: GoogleUpdate.exe

Detection ratio: 0/75

Security vendor	Result	Update
Trustlook	type-unsupported	20250529
SymantecMobileInsight	type-unsupported	20250124
BitDefenderKalx	type-unsupported	20250416
Avast-Mobile	type-unsupported	20250529
Bkav	undetected	20250529
Lionic	undetected	20250529
Elastic	undetected	20250528
MicroWorld-eScan	undetected	20250529
CTX	undetected	20250529
CAT-QuickHeal	undetected	20250528
Skyhigh	undetected	20250529
ALYac	undetected	20250529
Cylance	undetected	20250529
Zillya	undetected	20250529
Sangfor	undetected	20250526
K7AntiVirus	undetected	20250529
BitDefender	undetected	20250529
K7GW	undetected	20250529
CrowdStrike	undetected	20231026
Baidu	undetected	20190318
VirIT	undetected	20250529
Symantec	undetected	20250529
tehtris	undetected	20250529
ESET-NOD32	undetected	20250529
APEX	undetected	20250528

- **Noew.SAM:**
    - **Purpose:** The name does not correspond to any known legitimate file and appears suspicious. It could serve as a configuration file for the malware, a payload for data exfiltration, or another type of malicious executable.





Summary of Purposes:

The dropped files collectively suggest the following purposes:

- **Execution of Malicious Code:** Utilizing rundll32.exe to run potentially harmful DLLs.
- **Persistence and Updates:** Using Goopdate.dll to maintain the malware's presence on the system, potentially reinstalling itself or other components if removed.
- **Deceptive Operations:** GoogleTool.exe and similar files may trick users into believing they are interacting with legitimate software while conducting malicious activities.
- **Data Handling:** The presence of a file like Noew.SAM implies possible data storage or configuration, possibly for tracking user activities or system information.

Overall, these files would likely work together to ensure the malware remains persistent, operates stealthily, and possibly communicates with external servers or downloads additional payloads.

Explain the obfuscation technique used to cover the dropped files.

The code snippet demonstrates several obfuscation techniques to conceal the true nature and intentions of the dropped files:

- **Deceptive Naming:**
  The use of names like GoogleTool.exe and Goopdate.dll is a common obfuscation tactic. By mimicking legitimate software names, the malware can avoid detection by users and security software.
- **Dynamic Path Construction:**
  The code dynamically constructs file paths using SHGetSpecialFolderPathW to retrieve system-specific locations. This makes it harder for security software to identify where the files are being placed and what they are intended for, as the paths vary based on the system.
- **String Manipulation:**
  The code contains loops that manipulate byte values in memory (as seen with the

  array operations). Such manipulation can obfuscate the real purpose of the data being handled and complicate reverse engineering by obscuring the control flow.
- **Use of Legitimate Windows Functions:**
  By calling standard Windows API functions (like rundll32.exe), the malware can blend in with normal system operations, evading detection by some security mechanisms that may not flag legitimate system calls even when used maliciously.

- **Hidden Execution:**
  By simulating keyboard events and using legitimate processes, the malware can execute functions without drawing attention, making it appear as if user actions are triggering the behavior, further disguising its activities.

Summary:
These obfuscation techniques create an environment where the malware can operate unnoticed. The combination of deceptive file naming, dynamic pathing, and leveraging legitimate system functionality makes it challenging for users and security software to detect the presence and intentions of the dropped files. By obscuring their true purpose, the malware increases its chances of remaining undetected and effectively executing its harmful activities.

What is the purpose of creating 'Windows Message.lnk'?

Figure 3: File system and shortcut (.lnk) file setup used to achieve persistence via user deception.

Creating a shortcut file like Windows Message.lnk typically serves several potential purposes in the context of malware:

- **User Deception:**
  By naming the shortcut something benign or familiar, the malware attempts to trick users into clicking it. If users believe it's a legitimate system shortcut, they are less likely to question its presence or functionality.

- **Launching Malicious Payloads:**
  The .lnk file can be configured to execute a specific program or script when opened. In this case, it might run a malicious executable or script, effectively allowing the malware to spread or execute additional payloads without user awareness.

- **Persistence:**
  Creating a shortcut in a prominent location (like the desktop or startup folder) ensures the malware is easily accessible or automatically runs every time the user logs in, increasing the likelihood of repeated execution.

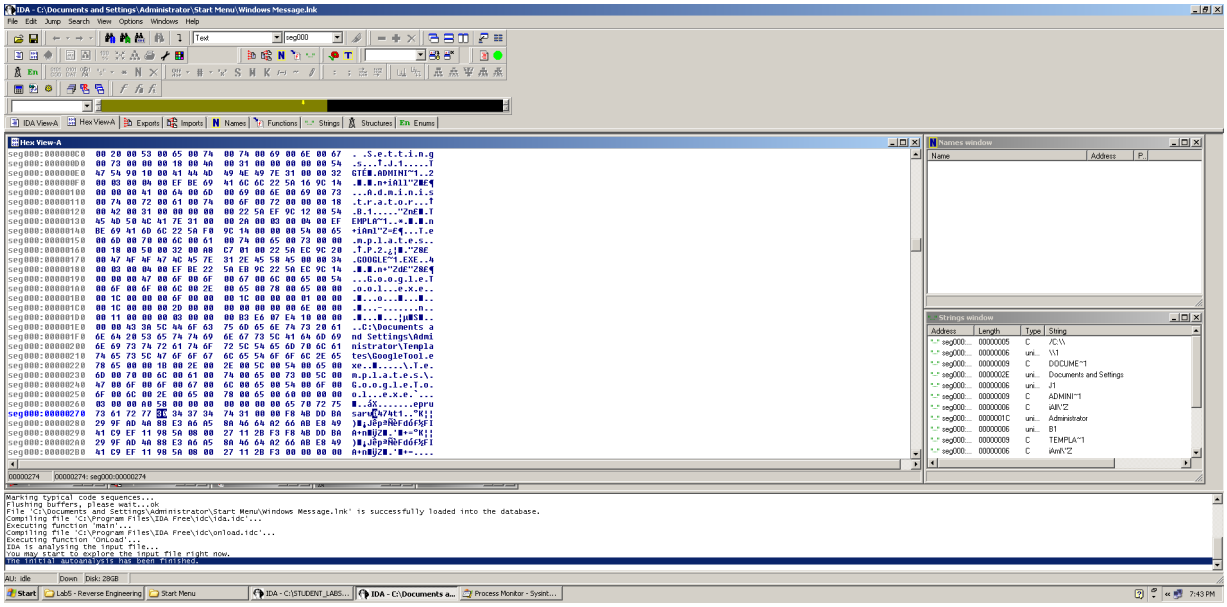
- **Redirection:**
  The shortcut might redirect to a malicious website or download additional malware when clicked, facilitating further infection or data exfiltration.

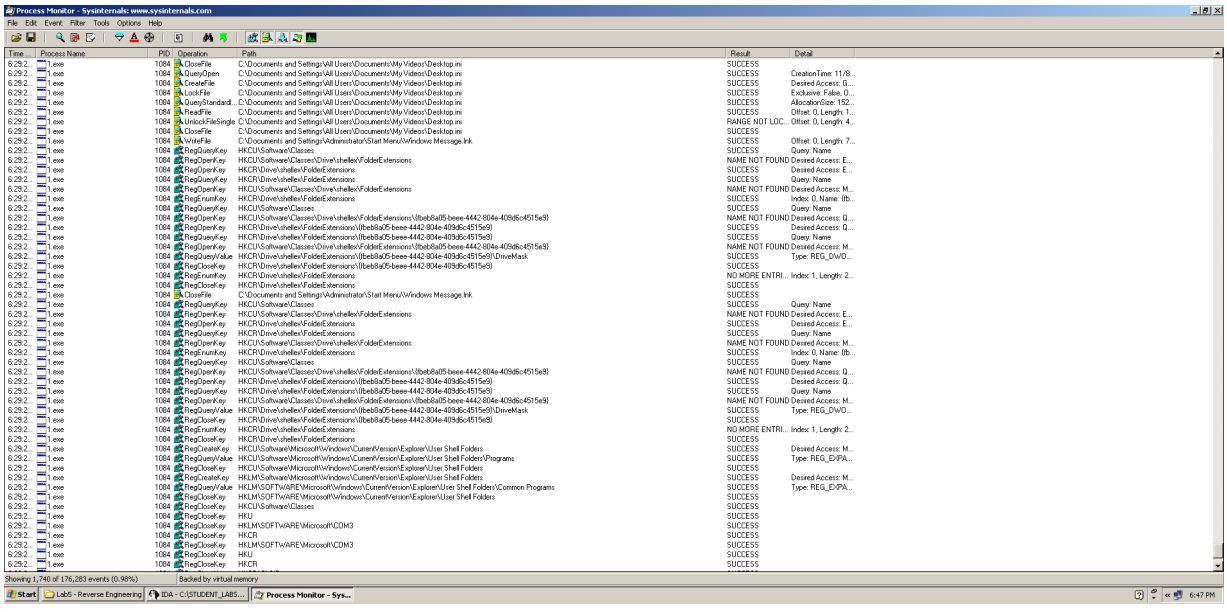
- **Social Engineering:**
  By naming the shortcut in a way that suggests it's important or relevant to the user

(like "Windows Message"), it leverages social engineering tactics to prompt the user to open it.

Summary:

The creation of Windows Message.lnk likely aims to deceive users, provide a means to execute malicious code, and enhance the persistence and stealth of the malware. By using familiar names and standard file types, malware authors increase the chances that their malicious activities go unnoticed.





Explain how the malicious payload is executed.

Execution Flow:

1. **Retrieve Special Folder Path:** The malware fetches a special folder path for file storage.
2. **Construct Paths:** It builds paths for malicious executables and the shortcut.
3. **Create Shortcut:** Generates Windows Message.lnk to facilitate execution.
4. **Simulate User Input:** Uses keybd_event to invoke commands automatically.
5. **Execute Malicious Payload:**GoogleTool.exe is run, loading Goopdate.dll via DLL side-loading.
6. **Ensure Persistence:** Maintains its presence and potential for further execution.

What is the obfuscation algorithm used to decode 'Noew.SAM'? Write pseudocode.

Obfuscation Algorithm: The obfuscation algorithm that decodes the string 'Noew.SAM' involves manipulating the byte order in memory to obfuscate its true representation. The specific segment of code appears to perform swaps between bytes, indicative of a simple obfuscation mechanism.

Pseudocode for the Obfuscation Algorithm: // Pseudocode for

the decoding (obfuscation) algorithm void

decode_obfuscation(char *input, char *output, int length) { for

(int i = 0; i < length; i += 4) { // Read four bytes from the

input char byte1 = input[i]; char byte2 = input[i +

1]; char byte3 = input[i + 2]; char byte4 = input[i + 3];

// Swap the bytes in a specific order

output[i] = byte3; // Place third byte first

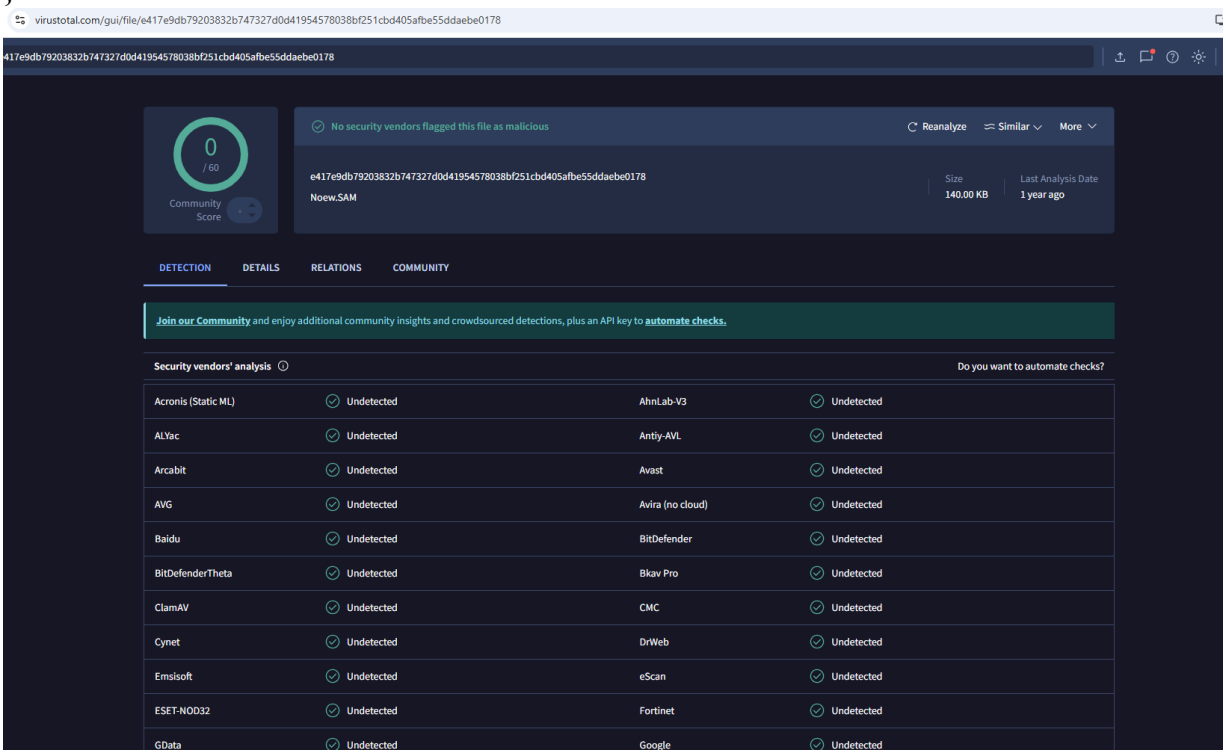
output[i + 1] = byte4; // Place fourth byte second

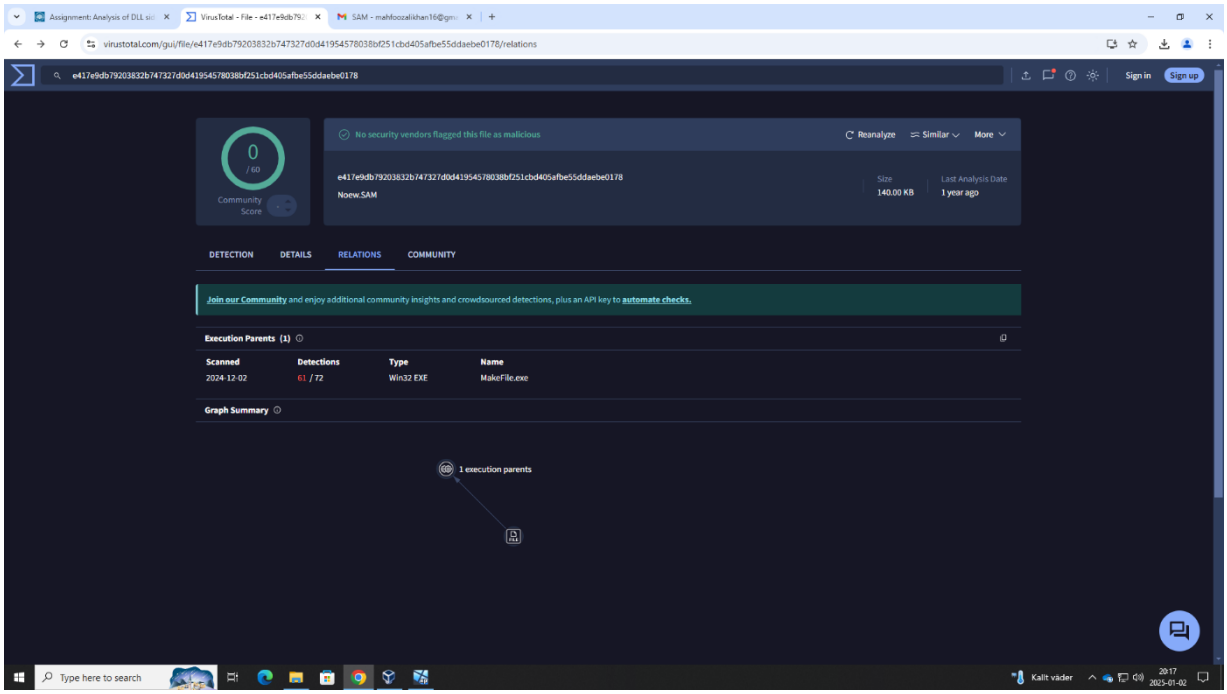
output[i + 2] = byte1; // Place first byte third

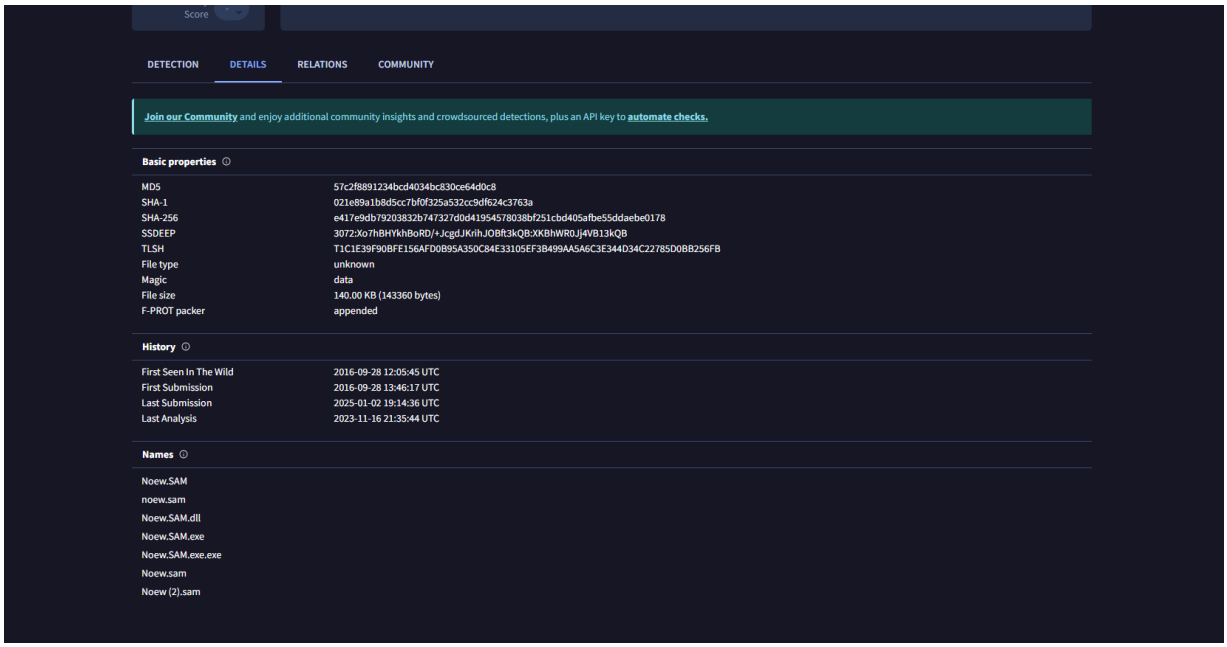
output[i + 3] = byte2; // Place second byte last

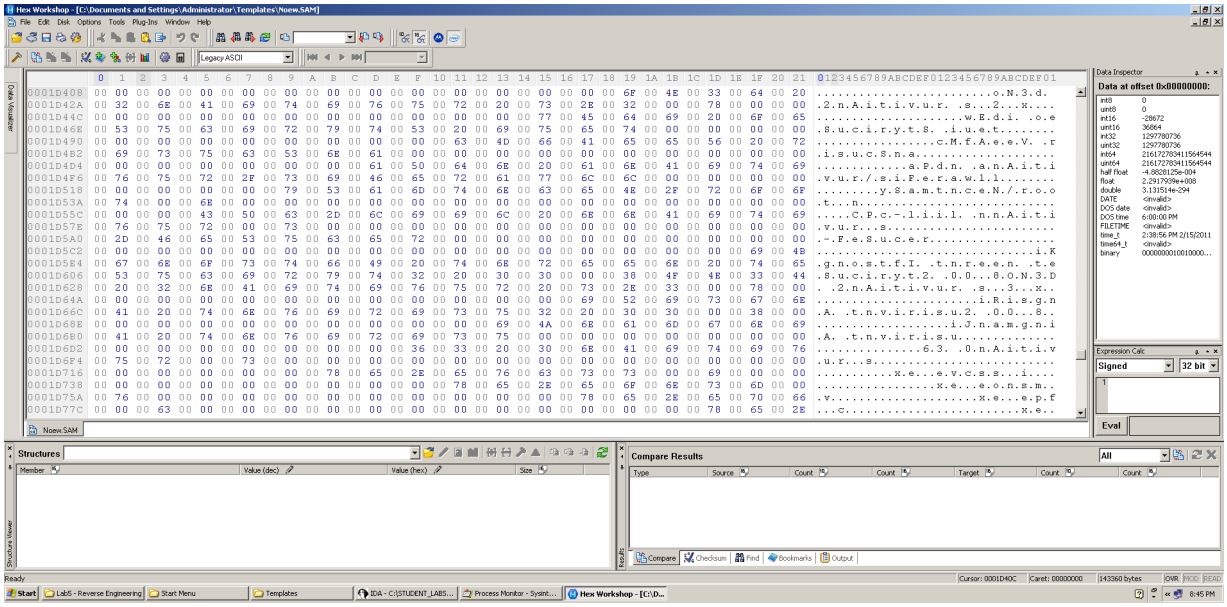
```
    }
}

// Main function void main() {     char obfuscated[] = "encoded_value"; // The original
obfuscated value of 'Noew.SAM'     char decoded[sizeof(obfuscated)];
decode_obfuscation(obfuscated, decoded, sizeof(obfuscated));
    // Output the decoded string
printf("Decoded value: %s\n", decoded);
}
```




virustotal.com/gui/file/e417e9db79203832b747327d0d41954578038bf251cbd405afbe55ddaebe0178/relations
e417e9db79203832b747327d0d41954578038bf251cbd405afbe55ddaebe0178
Sign in
Sign up
0
/ 60
Community Score
No security vendors flagged this file as malicious
Reanalyze
Similar
More
e417e9db79203832b747327d0d41954578038bf251cbd405afbe55ddaebe0178
Noew.SAM
Size
140.00 KB
Last Analysis Date
1 year ago
DETECTION
DETAILS
RELATIONS
COMMUNITY
Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.
Execution Parents (1)
Scanned
Detections
Type
Name
2024-12-02
61 / 72
Win32 EXE
MakeFile.exe
Graph Summary
1 execution parents
Type here to search
Kallt väder
2017
2025-01-02


Score
DETECTION
DETAILS
RELATIONS
COMMUNITY
Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.
Basic properties
MD5 57c2f8891234bcd4034bc830ce64d0c8
SHA-1 021e89a1b8d5cc7bf0f325a532cc9df624c3763a
SHA-256 e417e9db79203832b747327d0d41954578038bf251cbd405afbe55ddaebe0178
SSDEEP 3072:Xo7hBHYkhBoRD/+JcgdJKrihJOBft3kQB:XKBhWR0Jj4VB13kQB
TLSH T1C1E39F90BFE156AFD0B95A350C84E33105EF3B499AA5A6C3E344D34C22785D0BB256FB
File type unknown
Magic data
File size 140.00 KB (143360 bytes)
F-PROT packer appended
History
First Seen In The Wild 2016-09-28 12:05:45 UTC
First Submission 2016-09-28 13:46:17 UTC
Last Submission 2025-01-02 19:14:36 UTC
Last Analysis 2023-11-16 21:35:44 UTC
Names
Noew.SAM
noew.sam
Noew.SAM.dll
Noew.SAM.exe
Noew.SAM.exe.exe
Noew.sam
Noew (2).sam



DLL Side-Loading Technique in the Context of the Attack

Overview of DLL Side-Loading

What is DLL Side-Loading?

DLL side-loading is legitimate and executable used by attacker a target system by exploiting how Windows manages Dynamic Link Libraries (DLLs). The attacker tricks an application into loading a malicious DLL instead of the intended legitimate DLL. **How DLL Side-Loading Works**

1. **DLL Search Order**:
   When an application attempts to load a DLL, Windows searches for the DLL in various locations, including: ○ The application's directory. ○ The system directory. ○ The Windows directory. ○ Directories listed in the system's PATH environment variable.

2. **Malicious Placement**:
   If a malicious DLL with the same name as a legitimate one is placed in a directory that is searched before the legitimate DLL, the application will load the malicious DLL unknowingly.

3. **Execution of Malicious Code**:
   Once the application loads the malicious DLL, the code within that DLL is executed, allowing the attacker to carry out their objectives.

GoogleTool.exe is not malicious but it the malicious behavior is due to Goopdate.dll

Application of DLL Side-Loading in the Context of Your Code

In the context of the provided code and the directories specified (such as C:\Users\mahfooz\AppData\Roaming\Microsoft\Windows\Templates), the DLL sideloading technique can be seen in the following aspects:

1. **Path Construction**:
    - The code constructs paths for executables and DLLs, such as Goopdate.dll and GoogleTool.exe. This indicates that the attacker is ensuring these files reside in directories where the application expects to find them.
2. **Malicious File Naming**:
    - Using names like Goopdate.dll exploits the resemblance to legitimate software components, potentially tricking the user or system into executing the malicious version instead of the genuine one.
3. **Using Trusted Contexts**:
    - The code may leverage rundll32.exe to execute the DLL. Since rundll32.exe is a legitimate Windows utility used to run DLLs, its use can make the execution of the malicious payload less detectable.
4. **Special Folder Utilization**:
    - Calls to functions like SHGetSpecialFolderPathW retrieve paths to special folders (e.g., Application Data). These locations are often overlooked, making them ideal for hiding malicious DLLs.
5. **Dynamic File Manipulation**:
    - The assembly code dynamically appends paths and filenames, suggesting a strategy to create an environment where malicious files can be loaded without raising suspicion.

Attack Scenario Example

1. **Preparation**:
   The attacker creates a malicious DLL, such as Goopdate.dll, mimicking the name of a legitimate DLL that the target application is expected to load.

2. **Placement**:

   Using the constructed paths from the provided assembly code, the attacker places Goopdate.dll in the application's directory or a special folder where the application will find it during its DLL search process.
3. **Execution**:
   When the target application runs, it attempts to load Goopdate.dll. Due to the DLL side-loading technique, it loads the malicious version instead of the legitimate DLL.

4. **Payload Execution**:
   The malicious code within the DLL is executed, potentially granting the attacker remote access, exfiltrating sensitive data, or installing additional malware.

Countermeasures Against DLL Side-Loading

To defend against DLL side-loading attacks, organizations and users can implement several strategies:

- **Path Whitelisting**:
  Restrict applications to load DLLs only from verified and secure directories.

- **Code Signing**:
  Utilize digitally signed DLLs and enforce checks to verify the integrity of DLLs before loading them.

- **Monitor Application Behavior**:
  Deploy intrusion detection systems to monitor for unusual application behavior indicative of DLL side-loading.

- **Least Privilege**:
  Limit the permissions of applications to prevent them from executing unauthorized files.

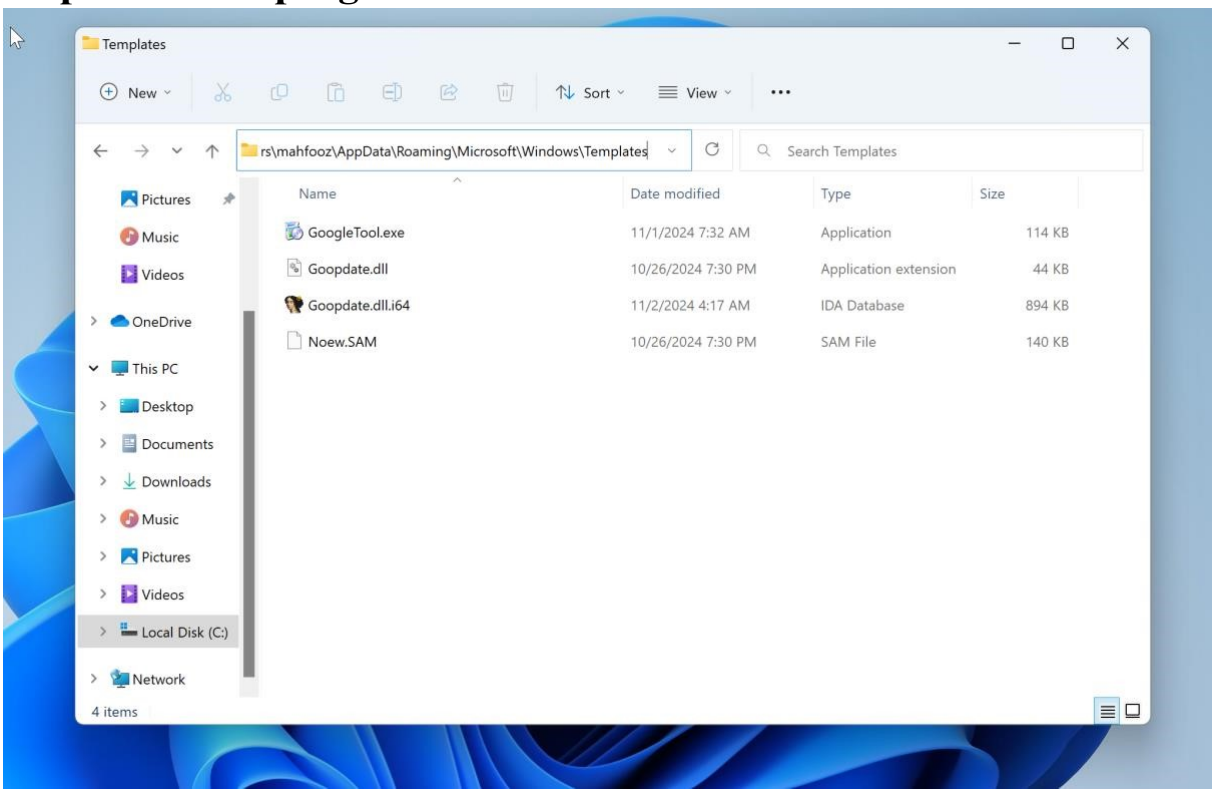
- **Security Software**:
  Employ security solutions capable of detecting and blocking suspicious DLL loading activities.

Conclusion

The DLL side-loading technique exploits the DLL search order in Windows to execute malicious code disguised as legitimate files. The provided assembly code exemplifies how attackers can manipulate paths and filenames to exploit this vulnerability, underscoring the need for robust security practices to mitigate such risks. The specified file paths indicate a

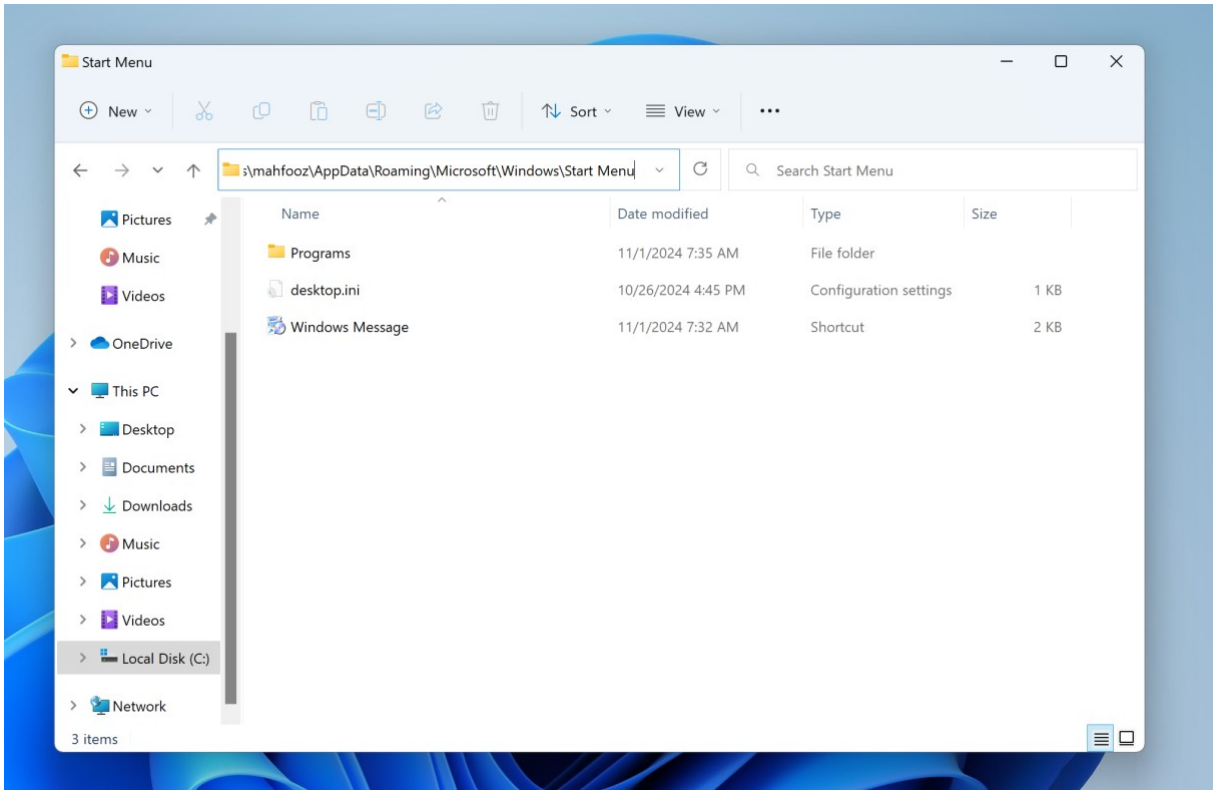
targeted approach to executing the attack within less scrutinized directories, highlighting the importance of vigilance and monitoring in security protocols.

file path where programs were stored



C:\Users\mahfooz\AppData\Roaming\Microsoft\Windows\Templates

It contains GoogleTool.exe Goopdate.dll Noew.SAm files in it



C:\Users\mahfooz\AppData\Roaming\Microsoft\Windows\Start Menu

It contains Windows Message Ink

References
Class notes c:\STUDENT_LABS\Lab5 - Reverse Engineering\"

IDA pro

Procmon